6 Prompt original y contexto de generación

El código se generó en Codex para escritorio con agentes de arquitectura y desarrollo. La familia del modelo fue GPT-5; la versión exacta no quedó visible en el registro exportado. El prompt inicial indicó las tecnologías, pero no nombró un estilo arquitectónico ni pidió compararlo antes de programar.

Prompt inicial conservado

```
Necesito realizar una aplicacion con javaScript en express y con react hooks donde podamos desarrollar el README que t
```

El pedido de React y Express determinó indirectamente una relación cliente-servidor. En cambio, la organización interna del backend quedó abierta. El agente arquitecto recomendó un monolito modular con capas y repositorios; el código generado conservó un monolito simple con separación técnica básica.

7 Consulta independiente sobre el código actual

La consulta se realizó en un contexto separado y se contrastó con el repositorio. La respuesta identifica el estilo implementado y distingue la arquitectura actual de la arquitectura objetivo.

Pregunta

```
¿Qué estilo arquitectónico emplea este código, cómo se manifiesta y cuál debió emplearse frente a los escenarios de ca
```

Respuesta documentada

El código emplea una arquitectura cliente-servidor: una SPA React consume una API REST Express. El backend es un monolito con una base SQLite compartida. Existe una separación técnica básica entre presentación, API y persistencia, pero no un monolito modular formal. Autenticación, hamburguesas, campañas y votos no están aislados mediante módulos con interfaces públicas; además, varios handlers HTTP ejecutan SQL directamente. Por ello tampoco hay una arquitectura hexagonal completa.

Componente	Evidencia en el repositorio	Interpretación
Cliente	client/src/App.jsx y client/src/api.js	SPA React que consume JSON por HTTP
Servidor	server/src/app.js	API REST Express; rutas, validación y reglas concentradas
Persistencia	server/src/db.js y SQLite	Esquema relacional; repositorio parcial
Despliegue	Un proceso Node y una base local	Monolito desplegado como unidad
Pruebas	server/test/api.test.js	Integración con SQLite en memoria, no puertos simulados

Tabla 1. Evidencia del estilo implementado en el repositorio.

Para el MVP actual, el monolito simple mantiene bajo el costo de construcción. Para incorporar pedidos y otras capacidades sin degradar la mantenibilidad, el sistema debería evolucionar hacia un monolito modular cliente-servidor. Cada dominio tendría controller, service y repository propios, y la persistencia se aislaría mediante puertos y adaptadores. Microservicios y eventos añadirían complejidad operativa desproporcionada para el alcance y el equipo.

8 Contraste de decisiones arquitectónicas

Aspecto	Estilo empleado	Estilo recomendado por la consulta	Decisión previa del equipo
Estilo principal	Monolito simple con capas técnicas	Monolito modular	Monolito modular
Distribución	SPA React + API REST + SQLite	Cliente-servidor	Cliente-servidor
Interior	Rutas y reglas juntas en app.js; SQL también en handlers	Controller-Service-Repository por dominio	Capas por módulo y puertos selectivos
Pruebas	Integración con SQLite en memoria	Reglas con repositorios falsos más integración real	Núcleo aislado de la base
Clasificación	Cliente-servidor indicado indirectamente; monolito simple espontáneo	Propuesto después de revisar código y escenarios	Decisión documentada antes de la consulta

Tabla 2. Contraste entre el código, la consulta y la decisión del equipo.

La diferencia es verificable: el diseño del equipo y la recomendación del asistente coinciden en el objetivo modular, pero el código no materializa todavía esas fronteras. Pedir tecnologías no fue suficiente para obtener la estructura interna esperada.

9 Experimento de reformulación

Se ejecutaron dos variantes aisladas con el mismo tipo de asistente y el mismo alcance: un esqueleto del módulo Votación. La primera pidió estudiar alternativas; la segunda impuso el estilo y reglas de dependencia.

Variante A estudio previo

```
Antes de generar código, compara capas, monolito modular, microservicios y hexagonal frente a 500 votos concurrentes, p
```

Resultado. El asistente eligió un monolito modular con arquitectura hexagonal dentro de Votación. Propuso módulos de negocio, un puerto VoteRepository, un caso de uso CastVote, adaptador PostgreSQL, pruebas con un repositorio falso y una prueba de carga. También aclaró que p95 y cero votos perdidos deben medirse; la arquitectura por sí sola no los demuestra.

Variante B estilo explícito

```
Genera el módulo Votación de Secret Bite con un monolito modular cliente-servidor. Dentro del módulo usa Controller ->
```

Resultado. El asistente siguió la estructura pedida sin volver a debatir el estilo. Produjo una API pública del módulo, carpetas http, application, ports, infrastructure y domain, y separó adaptadores PostgreSQL y SQLite. La salida fue más determinista y más fácil de revisar contra reglas de dependencias.

Criterio	Original	Variante A estudia	Variante B explícita
Elección	Monolito simple espontáneo	Elige modular + hexagonal	Aplica modular + capas + puerto
Fronteras	No verificables	Propone módulos por negocio	Nombra carpetas y dependencias permitidas
SQL en handlers	Sí	Lo evita mediante repositorio	Lo prohíbe expresamente
Pruebas sin DB	No; usa SQLite en memoria	FakeVoteRepository	InMemoryVoteRepository
Riesgo	Desviación respecto del diseño	Puede elegir otra opción si cambian supuestos	Puede obedecer un estilo inadecuado si el equipo se equivoca

Tabla 3. Comparación de las tres generaciones.

10 Conclusiones y recomendación

- El asistente eligió espontáneamente un monolito simple porque el prompt original fijó tecnologías y alcance, pero no fronteras internas.
- Cuando se le pidió estudiar alternativas, eligió una combinación coherente con la decisión previa del equipo y explicó los compromisos.
- Cuando el estilo se indicó de forma explícita, la estructura fue más consistente y verificable: ningún controller ejecutó SQL y las reglas se probaron mediante puertos.
- La mejor estrategia combina ambos enfoques: pedir primero una comparación breve basada en escenarios y, después de aprobar la decisión, generar con reglas arquitectónicas explícitas y criterios de aceptación.

Recomendación de prompt

```
Analiza primero capas, monolito modular, microservicios y hexagonal frente a los escenarios de calidad adjuntos. Prese
```

Limitaciones del experimento

- Se evaluó un solo sistema y una sola familia de modelo.
- Las respuestas de un modelo generativo pueden variar entre ejecuciones.

- Las variantes produjeron esqueletos, no una segunda aplicación completa.
- El objetivo de 500 votos y p95 menor de dos segundos no se verificó con una prueba de carga en infraestructura productiva.
- Pedidos sigue siendo una capacidad futura; no está implementada en el MVP revisado.

11 Registro de uso de inteligencia artificial

Herramienta	Uso	Versión registrada	Verificación humana
Codex escritorio	Inspección del repositorio y desarrollo del MVP	Familia GPT-5; versión exacta no visible	Se ejecutaron build, seed y cuatro pruebas
Agente documentador	Comparó guía, PDF y código	Misma plataforma; versión exacta no visible	Se contrastaron rutas, esquema y pruebas
Dos ejecuciones aisladas	Variantes A y B del experimento	Mismo tipo de agente	Se compararon estructura, dependencias y limitaciones

Prompt	Cambio principal	Motivo
Original	Solo tecnologías y alcance	Construir el MVP
Consulta	Entrega el código y pregunta estilo usado/debido	Separar evidencia del código de la decisión previa
Variante A	Obliga a comparar antes de generar	Evaluar si la IA razona la elección
Variante B	Fija estilo y reglas verificables	Evaluar obediencia y consistencia estructural
Mejorado	Dos etapas: decisión aprobada y generación restringida	Reducir decisiones implícitas y facilitar revisión

Tabla 4. Evolución de los prompts principales.

12 Ajustes pendientes antes de entregar

- Completar institución, asignatura, docente, grupo y fecha en la portada.
- Confirmar los nombres y tildes de todos los integrantes y el tamaño real del equipo.
- Decidir si Pedidos pertenece al alcance evaluado o se presenta únicamente como evolución futura.
- Ajustar las métricas de los escenarios al ERS aprobado y no afirmar alta disponibilidad ni balanceo como funciones implementadas.
- Renderizar los diagramas de cada estilo en la primera parte; conservar su código Mermaid en anexos.

- Agregar capítulo o página de libros y norma ISO cuando el equipo tenga acceso a los ejemplares consultados.

Referencias complementarias

- Cockburn, A. (2005). Hexagonal architecture. https://alistair.cockburn.us/hexagonal-architecture/
- Fowler, M. (2017, 7 de febrero). What do you mean by event-driven? https://martinfowler.com/articles/201701-event-driven.html
- Lewis, J., & Fowler, M. (2014, 25 de marzo). Microservices. https://martinfowler.com/articles/microservices.html
- Microsoft. (s. f.). Common web application architectures. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures
- Svrtan, D., & Makagon, S. (2020, 10 de marzo). Ready for changes with hexagonal architecture. Netflix Technology Blog.
- Westeinde, K. (2019, 21 de febrero). Deconstructing the monolith. Shopify Engineering.